

Javascript

```

    dbColumnName = <code_set_nm>{codeSetName}</code_set_nm>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <code>{fieldData}</code>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <code_desc_txt>{fieldData}</code_desc_txt>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <code_short_desc_txt>{fieldData}</code_short_desc_txt>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <field_path>{pathHL7}</field_path>;
    tempEle.appendChild(dbColumnName);
    xmlNodeName.appendChild(tempEle);
  }
}

//FUNCTION: fnCreateXMLLocalCheckEntry
//Function to create an XML node for checking LOINC codes and local codes.
function fnCreateXMLLocalCheckEntry (pathHL7, xmlNodeName) {
  var dbColumnName;
  var fieldData = next.getField(pathHL7);
  if (fieldData != null && fieldData != '') {
    tempEle = new XML(<query_record></query_record>);
    dbColumnName = <lab_id>{next.getField('MSH/SendingFacility/UniversalID')}</lab_id>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <lab_test_cd>{fieldData}</lab_test_cd>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <field_path>{pathHL7}</field_path>;
    tempEle.appendChild(dbColumnName);
    xmlNodeName.appendChild(tempEle);
  }
}

//FUNCTION: fnCreateXMLLOINCEntry
//Function to create an XML node for checking LOINC codes in
// LOINC_condition and LOINC_code tables.
function fnCreateXMLLOINCEntry (pathHL7, xmlNodeName) {
  var dbColumnName;
  var fieldData = next.getField(pathHL7);
  if (fieldData != null && fieldData != '') {
    tempEle = new XML(<query_record></query_record>);
    dbColumnName = <loinc_cd>{fieldData}</loinc_cd>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <field_path>{pathHL7}</field_path>;
    tempEle.appendChild(dbColumnName);
    xmlNodeName.appendChild(tempEle);
  }
}

//FUNCTION: fnCreateXMLSNOMEDEntry
//Function to create an XML node for checking LOINC codes in
// SNOMED_condition and SNOMED_code tables.
function fnCreateXMLSNOMEDEntry (pathHL7, xmlNodeName) {
  var dbColumnName;

```

Javascript

```

var fieldData = next.getField(pathHL7);
if (fieldData != null && fieldData != '') {
  tempEle = new XML(<query_record></query_record>);
  dbColumnName = <snomed_cd>{fieldData}</snomed_cd>;
  tempEle.appendChild(dbColumnName);
  dbColumnName = <field_path>{pathHL7}</field_path>;
  tempEle.appendChild(dbColumnName);
  xmlNodeName.appendChild(tempEle);
}
}

//
=====
// _____ FUNCTIONS (end
)
//
=====

//
=====
// _____ Build Output XML file (start
)
//
=====
//Create an XML output file to be used to query the NBS_SRTE tables to validate codes or text.

//MSH
fnCreateXMLEntry ('MSH/SendingFacility/UniversalIDType', nodeCode_value_general, 'EI_AUTH');

//----- PATIENT_RESULT group -----
for (i=0; i < next.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT['+i+']/PATIENT/PID';

  //----- PID segment -----
  //PatientIdentifierList
  for (j=0; j < next.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/AssigningAuthority/UniversalIDType',
nodeCode_value_general, 'EI_TYPE');
    if ((next.getField(PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode') != null) &&
(next.getField(PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode') != '')) {
      fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode', nodeCode_value
_general, 'EI_TYPE');
    }
    fnCreateXMLEntry (PID + '/PatientIdentifierList['+j+']/AssigningFacility/UniversalIDType',
nodeCode_value_general, 'EI_TYPE');
  }
  //PatientIdentifierList (end)

  //PatientName - ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (PID + '/PatientName[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
  fnCreateXMLEntry (PID + '/PatientName[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
  fnCreateXMLEntry (PID + '/PatientName[0]/Degree_DEPRECATED', nodeCode_value_general, 'P_NM
_DEG');
  fnCreateXMLEntry (PID + '/PatientName[0]/NameTypeCode', nodeCode_value_general, 'P_NM_USE');
  //PatientName (end)

```

Javascript

```

fnCreateXMLEntry (PID + '/AdministrativeSex', nodeCode_value_general, 'SEX');

//Race
for (j=0; j < next.getRepeatCount(PID + '/Race'); j++) {
  fnCreateXMLEntry (PID + '/Race['+j+']/Identifier', nodeV_Race_code, 'P_RACE');
}
//Race (end)

//PatientAddress - ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/PatientAddress[0]/StateOrProvince', nodeState_code, 'STATE_CCD');
fnCreateXMLEntry (PID + '/PatientAddress[0]/Country', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (PID + '/PatientAddress[0]/AddressType', nodeCode_value_general, 'EL_TYPE_PST_PAT');
fnCreateXMLEntry (PID + '/PatientAddress[0]/CountyParishCode', nodeV_State_county_code_value, 'COUNTY_CCD');
//PatientAddress (end)

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/HomePhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general, 'EL_USE');
fnCreateXMLEntry (PID + '/HomePhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_general, 'EI_TYPE');
fnCreateXMLEntry (PID + '/BusinessPhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general, 'EL_USE');
fnCreateXMLEntry (PID + '/BusinessPhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_general, 'EI_TYPE');

//ODS Comment: Not treated as repeatable.
fnCreateXMLEntry (PID + '/PrimaryLanguage[0]/Identifier', nodeLanguage_code, 'P_LANG');

fnCreateXMLEntry (PID + '/MaritalStatus/Identifier', nodeCode_value_general, 'P_MARITAL');
fnCreateXMLEntry (PID + '/PatientAccountNumber/IdentifierTypeCode', nodeCode_value_general, 'EI_TYPE');
fnCreateXMLEntry (PID + '/DriversLicenseNumberPatient/IssuingStateProvinceCountry', nodeCode_value_general, 'EI_AUTH');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (PID + '/MothersIdentifier[0]/IdentifierTypeCode', nodeCode_value_general, 'EI_TYPE');

fnCreateXMLEntry (PID + '/EthnicGroup/Identifier', nodeCode_value_general, 'P_ETHN');
fnCreateXMLEntry (PID + '/BirthPlace', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (PID + '/MultipleBirthIndicator', nodeCode_value_general, 'YNU');
fnCreateXMLEntry (PID + '/PatientDeathIndicator', nodeCode_value_general, 'YNU');
//----- PID segment (end) -----

//----- NK1 segment -----
for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/PATIENT/NK1'); j++) {
  var NK1 = 'PATIENT_RESULT['+i+']/PATIENT/NK1['+j+']';
  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (NK1 + '/Name[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
  fnCreateXMLEntry (NK1 + '/Name[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
  fnCreateXMLEntry (NK1 + '/Name[0]/Degree_DEPRECATED', nodeCode_value_general, 'P_NM_DEG');
  fnCreateXMLEntry (NK1 + '/Name[0]/NameTypeCode', nodeCode_value_general, 'P_NM_USE');

  fnCreateXMLEntry (NK1 + '/Relationship/Identifier', nodeCode_value_general, 'RL_TYPE');
}

```

Javascript

```

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (NK1 + '/Address[0]/StateOrProvince', nodeState_code, 'STATE_CCD');
fnCreateXMLEntry (NK1 + '/Address[0]/Country', nodeCode_value_general, 'PSL_CNTRY');
fnCreateXMLEntry (NK1 + '/Address[0]/AddressType', nodeCode_value_general, 'EL_TYPE_PST
PAT');
fnCreateXMLEntry (NK1 + '/Address[0]/CountyParishCode', nodeV_State_county_code_value,
'COUNTY_CCD');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (NK1 + '/PhoneNumber[0]/TelecommunicationUseCode', nodeCode_value_general,
'EL_USE');
fnCreateXMLEntry (NK1 + '/PhoneNumber[0]/TelecommunicationEquipmentType', nodeCode_value_gen
eral, 'EI_TYPE');
}
//----- NK1 segment (end) -----

//----- ORDER OBSERVATION group -----
for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) {
  var ORC = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/ORC';
  var OBR = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBR';

  //----- ORC segment -----
  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingFacilityName[0]/OrganizationNameTypeCode', nodeCode_value
_general, 'O_NM_USE');

  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/StateOrProvince', nodeState_code, 'STATE
_CCD');
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/Country', nodeCode_value_general, 'PSL
_CNTRY');
  fnCreateXMLEntry (ORC + '/OrderingFacilityAddress[0]/CountyParishCode', nodeV_State_county
_code_value, 'COUNTY_CCD');

  //ODS Comment: Presently NBS stores only one value
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/StateOrProvince', nodeState_code, 'STATE
_CCD');
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/Country', nodeCode_value_general, 'PSL
_CNTRY');
  fnCreateXMLEntry (ORC + '/OrderingProviderAddress[0]/CountyParishCode', nodeV_State_county
_code_value, 'COUNTY_CCD');
  //----- ORC segment (end) -----

  //----- OBR segment -----
  //-----OBR-4-----
  //Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.
  if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'L') {
    fnCreateXMLLocalCheckEntry (OBR + '/UniversalServiceIdentifier/Identifier', nodeLabtest
_loinc);
  }
  if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'L') {

```

Javascript

```

    fnCreateXMLLocalCheckEntry (OBR + '/UniversalServiceIdentifier/AlternateIdentifier', node
Labtest_loinc);
}

//Check LOINC codes against LOINC_condition and LOINC_code tables.
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'LN') {
    var tempPath = OBR + '/UniversalServiceIdentifier/Identifier';
    fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
    fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
}
if (next.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'LN')
{
    var tempPath = OBR + '/UniversalServiceIdentifier/AlternateIdentifier';
    fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
    fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
}
//-----OBR-4 (end)-----

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Suffix', nodeCode_value_general, 'P_NM
_SFX');
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Prefix', nodeCode_value_general, 'P_NM
_PFX');
fnCreateXMLEntry (OBR + '/CollectorIdentifier[0]/Degree_DEPRECATED', nodeCode_value_general,
'P_NM_DEG');

fnCreateXMLEntry (OBR + '/SpecimenActionCode', nodeCode_value_general, 'M_HANDLING');
fnCreateXMLEntry (OBR + '/DangerCode/Identifier', nodeCode_value_general, 'M_RISK');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/SpecimenSource[0]/SpecimenSourceNameOrCode/Identifier', nodeCode
_value_general, 'SPECMN_SRC');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Suffix', nodeCode_value_general, 'P_NM_SFX');
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Prefix', nodeCode_value_general, 'P_NM_PFX');
fnCreateXMLEntry (OBR + '/OrderingProvider[0]/Degree_DEPRECATED', nodeCode_value_general, 'P
_NM_DEG');

fnCreateXMLEntry (OBR + '/ResultStatus', nodeCode_value_general, 'ACT_OBJ_ST');

for (k=0; k < next.getRepeatCount(OBR + '/ResultCopiesTo'); k++) {
    fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Suffix', nodeCode_value_general, 'P_NM
_SFX');
    fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Prefix', nodeCode_value_general, 'P_NM
_PFX');
    fnCreateXMLEntry (OBR + '/ResultCopiesTo['+k+']/Degree_DEPRECATED', nodeCode_value_general,
'P_NM_DEG');
}

for (k=0; k < next.getRepeatCount(OBR + '/ReasonForStudy'); k++) {
    fnCreateXMLEntry (OBR + '/ReasonForStudy['+k+']/Identifier', nodeCode_value_general, 'OBS
_REASON');
}

fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Suffix', nodeCode_value_general, 'P
_NM_SFX');

```

Javascript

```

    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Prefix', nodeCode_value_general, 'P
_NM_PFX');
    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/Degree_DEPRECATED', nodeCode_value
_general, 'P_NM_DEG');
    fnCreateXMLEntry (OBR + '/PrincipalResultInterpreter/Name/AssigningAuthorityNamespaceID',
nodeCode_value_general, 'EI_AUTH');
    //----- OBR segment (end) -----

    //----- OBSERVATION group -----

    for (k=0; k < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVA
TION'); k++) {
        var OBX = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION['+k+']/OBX';
        //----- OBX segment -----

        //-----OBX-3-----
        //Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.

        if (next.getField(OBX + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'L') {
            fnCreateXMLLocalCheckEntry (OBX + '/UniversalServiceIdentifier/Identifier', nodeLabtest
_loinc);
        }
        if (next.getField(OBX + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'L')
        {
            fnCreateXMLLocalCheckEntry (OBX + '/UniversalServiceIdentifier/AlternateIdentifier', node
Labtest_loinc);
        }

        //Check LOINC codes against LOINC_condition and LOINC_code tables.
        if (next.getField(OBX + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'LN') {
            var tempPath = OBX + '/UniversalServiceIdentifier/Identifier';
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
        }
        if (next.getField(OBX + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') == 'LN')
        {
            var tempPath = OBX + '/UniversalServiceIdentifier/AlternateIdentifier';
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_code);
            fnCreateXMLLOINCEntry (tempPath, nodeLOINC_condition);
        }
        //-----OBX-3 (end)-----

        //-----OBX-5-----
        //Handle OBX-5 variations based on the data type in OBX-2
        var OBX5DataType = next.getField(OBX + '/ValueType');
        if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
            var OBX5CodingSys = next.getField(OBX + '/ObservationValue/NameOfCodingSystem');
            if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
                fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/Identifier', nodeSnomed_code);
                fnCreateXMLSNOMEDEntry (OBR + '/ObservationValue/Identifier', nodeSnomed_condition);
            }
            var OBX5AltCodingSys = next.getField(OBX + '/ObservationValue/NameOfAlternateCodingSystem
');
            if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
                fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed_code);
            }
        }
    }

```

Javascript

```

        fnCreateXMLSNOMEDEntry (OBX + '/ObservationValue/AlternateIdentifier', nodeSnomed_con
dition);
    }
}
if (OBX5DataType == 'SN') {
    fnCreateXMLEntry (OBX + '/ObservationValue/Comparator', nodeCode_value_general, 'OV_NUM
_CMP');
    fnCreateXMLEntry (OBX + '/ObservationValue/SeparatorSuffix', nodeCode_value_general, 'OV
_NUM_SEP');
}
//-----OBX-5 (end)-----

fnCreateXMLEntry (OBX + '/Units/Identifier', nodeCode_value_general, 'UNIT_ISO');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBX + '/AbnormalFlags[0]/Identifier', nodeCode_value_general, 'OBS_INTRP
');

fnCreateXMLEntry (OBX + '/ObservationResultStatus', nodeCode_value_general, 'ACT_OBJ_ST');
fnCreateXMLEntry (OBX + '/ProducersReference/NameOfCodingSystem', nodeCode_value_general,
'EI_AUTH');

//ODS Comment: Presently NBS stores only one value
fnCreateXMLEntry (OBX + '/ObservationMethod[0]/Identifier', nodeCode_value_general, 'OBS
_METH');

fnCreateXMLEntry (OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', nodeCode_value
_general, 'EI_AUTH');

    fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/StateOrProvince', nodeState_code,
'STATE_CCD');
    fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/Country', nodeCode_value_general,
'PSL_CNTRY');
    fnCreateXMLEntry (OBX + '/PerformingOrganizationAddress/CountyParishCode', nodeV_State
_county_code_value, 'COUNTY_CCD');
    //----- OBX segment (end) -----
}
//----- OBSERVATION group (end) -----

} // -----(END) for (j=0; j < next.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVA
TION'); j++) (END)
//--- ORDER_OBSERVATION group (end) ---

} // -----(END) for (i=0; i < next.getRepeatCount('PATIENT_RESULT'); i++) (END)
//--- PATIENT_RESULT group (end) ---

//
=====
//_____ Build Output XML file (end
)

```

Javascript

```
//
=====

dbInsertsXML.appendChild(nodeCode_value_general);
dbInsertsXML.appendChild(nodeV_Race_code);
dbInsertsXML.appendChild(nodeLanguage_code);
dbInsertsXML.appendChild(nodeV_State_county_code_value);
dbInsertsXML.appendChild(nodeState_code);
dbInsertsXML.appendChild(nodeLabtest_loinc);
dbInsertsXML.appendChild(nodeLOINC_condition);
dbInsertsXML.appendChild(nodeLOINC_code);
dbInsertsXML.appendChild(nodeSnomed_condition);
dbInsertsXML.appendChild(nodeSnomed_code);

next.text = dbInsertsXML;
```

Destinations

Name	Page
 db NBS_SRTE Query Results..... Database Message Extraction	379



js Extract NBS_SRTE Errors

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: In this JavaScript filter, the code traverses each segment and ordered group according to the message definition ECLIR_ELOR_ORU_2.3.1_NBS_validation.s3d for an ORU – Unsolicited Observation Message (Event R01).

A message collector is used to input 2 messages into the filter.

The first message is the output XML file received from the *db NBS_SRTE Query Results* Database Lookup filter.

The second message is the original HL7 v2 ELR message.

Specific fields in the message that should have a valid value or code in a code set within a corresponding table in the NBS_SRTE database are searched for in the XML file received from the *db NBS_SRTE Query Results* Database Lookup filter. If an element does not exist for the specific field, this data, along with other relevant fields, is written to an XML flat file that follows the ECLIR - Error.xsd format for use to write these errors to the ECLIR database in the *ECLIR - ELR Errors* route.

Javascript

```
//
=====
//_____INITIAL VARIABLES (start
)_____
//
=====

var next = output.append(input[0]);
var ELRMessage = input[1];
```

Javascript

```

var dbInsertsXML = new XML(<message></message>);
var tempEle;

//
=====
//_____INITIAL VARIABLES (end)
)_____
//
=====

//
=====
//_____FUNCTIONS (start)
)_____
//
=====

//FUNCTION: fnCreateErrorMsg
//Function to create the exact error message to be written to the ELR_ERROR table.
function fnCreateErrorMsg (pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, invalidType) {
  var errorMsg = '';
  switch (invalidType) {
    case 'Invalid_code':
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '" for code set name "' + codeSetName + '".';
      break;
    case 'Invalid_LOINC_code':
    case 'Invalid_SNOMED_code':
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '".';
      break;
    case 'Invalid_code_from_text':
      errorMsg += 'A code equivalent to field value "' + ELRMessage.getField(pathHL7) + '" in
        field ' + fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found
        in NBS_SRTE table "' + tblNBS_SRTE + '".';
      break;
    case 'Invalid_local_code':
      errorMsg += 'The local code "' + ELRMessage.getField(pathHL7) + '" in field ' + field
        Notation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') could not be converted to a
        LOINC code from the NBS_SRTE table "' + tblNBS_SRTE + '".';
      break;
    case 'Condition_not_found_for_LOINC':
    case 'Condition_not_found_for_SNOMED':
      errorMsg += 'The ' + tblNBS_SRTE.substr(0, tblNBS_SRTE.indexOf('_')) + ' code "' + ELRM
        essage.getField(pathHL7) + '" in field ' + fieldNotation + ' (' + pathHL7.substr(pathHL7.last
        IndexOf('/')+1) + ') did not have a matching condition in the NBS_SRTE table "' + tblNBS_SRTE +
        '".';
      break;
    default:
      errorMsg += 'The code "' + ELRMessage.getField(pathHL7) + '" in field ' + fieldNotation +
        ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') was not found in the NBS_SRTE table "'
        + tblNBS_SRTE + '" for code set name "' + codeSetName + '".';
      break;
  }
}

```

Javascript

```

}
return errorMsg;
}

//FUNCTION: fnCheckIfRecordReturned
//Function to check if a record was returned for the value in the field.
function fnCheckIfRecordReturned (requiredField, pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, errorCategory, errorTypeDetail) {
    //If the field is required, check for returned record regardless of value or
    // if the field is not required, check for a returned record if valued.
    if (requiredField || (ELRMessage.getField(pathHL7) != null && ELRMessage.getField(pathHL7) != '')) {
        var errorMsg = fnCreateErrorMsg (pathHL7, tblNBS_SRTE, codeSetName, fieldNotation, errorTypeDetail);

        //Check to see if a code was found in the NBS_SRTE.
        // If there is no existing node, the code was not found and an error is recorded.
        if (tblNBS_SRTE == 'Labtest_loinc') {
            if (next.getField("/message/" + tblNBS_SRTE + "[field_path='" + pathHL7 + "']/LocalToLOINC") == null) {
                fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
            }
            else {
                if (next.getField("/message/Labtest_loinc/CheckLOINC_code[field_path='" + pathHL7 + "']/loinc_cd") == null) {
                    errorMsg = fnCreateErrorMsg (pathHL7, 'LOINC_code', codeSetName, fieldNotation, 'Invalid LOINC_code');
                    fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
                }
                if (next.getField("/message/Labtest_loinc/CheckLOINC_condition[field_path='" + pathHL7 + "']/loinc_cd") == null) {
                    errorMsg = fnCreateErrorMsg (pathHL7, 'LOINC_condition', codeSetName, fieldNotation, 'Condition_not_found_for_LOINC');
                    fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
                }
            }
        }
        else {
            if (next.getField('/message/' + tblNBS_SRTE + '[field_path="' + pathHL7 + '"]/field_path') == null) {
                fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg);
            }
        }
    }
}

//FUNCTION: fnCreateErrorRecord
//Function to create an error record in the output XML to be saved to ELR_ERROR table.
function fnCreateErrorRecord (pathHL7, fieldNotation, errorCategory, errorMsg) {
    tempEle = new XML(<errorRecord></errorRecord>);
    dbColumnName = <VALIDATION_TYPE>{'NBS'}</VALIDATION_TYPE>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <ELR_MESSAGE_ID>{next.getProperty('ELR_MESSAGE_ID')}</ELR_MESSAGE_ID>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <ERROR_CATEGORY>{errorCategory}</ERROR_CATEGORY>;
    tempEle.appendChild(dbColumnName);
}

```

Javascript

```

dbColumnName = <PROCESSING_STEP>{'Validation: ECLIR - ELR Validation (Route):js Extract NBS
_SRTE Errors (Filter)'}</PROCESSING_STEP>;
tempEle.appendChild(dbColumnName);
dbColumnName = <FIELD>{fieldNotation}</FIELD>;
tempEle.appendChild(dbColumnName);
dbColumnName = <FIELD_PATH>{pathHL7}</FIELD_PATH>;
tempEle.appendChild(dbColumnName);
if (pathHL7 != null && pathHL7 != '') {
  if (ELRMessage.getField(pathHL7) != null) {
    dbColumnName = <FIELD_VALUE>{ELRMessage.getField(pathHL7)}</FIELD_VALUE>;
    tempEle.appendChild(dbColumnName);
  }
}
dbColumnName = <ERROR_MSG>{errorMsg}</ERROR_MSG>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE>{'NBS_SRTE CONFORMANCE'}</ERROR_TYPE>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE_DETAIL>{'Invalid value'}</ERROR_TYPE_DETAIL>;
tempEle.appendChild(dbColumnName);
dbInsertsXML.appendChild(tempEle);
}

//
=====
// _____ FUNCTIONS (end
)
//
=====

//
=====
// _____ Build Output XML file (start
)
//
=====
//Create an XML output file to be used to create error records in the ELR_ERROR table if codes
are not valid.

//----- MSH segment -----
fnCheckIfRecordReturned (true, 'MSH/SendingFacility/UniversalIDType', 'Code_value_general', 'EI
_AUTH', 'MSH-4.3', 'WARNING', 'Invalid_code');

//----- PATIENT_RESULT group -----
for (i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT['+i+']/PATIENT/PID';

  //----- PID segment -----
  //PatientIdentifierList
  for (j=0; j < ELRMessage.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/AssigningAuthority/Uni
versalIDType', 'Code_value_general', 'EI_TYPE', 'PID-3.4.3', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/IdentifierTypeCode',
'Code_value_general', 'EI_TYPE', 'PID-3.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (true, PID + '/PatientIdentifierList['+j+']/AssigningFacility/Uni
versalIDType', 'Code_value_general', 'EI_TYPE', 'PID-3.6.3', 'WARNING', 'Invalid_code');
  }
}

```

Javascript

```

}
//PatientIdentifierList (end)

//PatientName - ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Suffix', 'Code_value_general', 'P_NM_SFX', 'PID-5.4', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Prefix', 'Code_value_general', 'P_NM_PFX', 'PID-5.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'PID-5.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientName[0]/NameTypeCode', 'Code_value_general', 'P_NM_USE', 'PID-5.7', 'INFO', 'Invalid_code');
//PatientName (end)

fnCheckIfRecordReturned (false, PID + '/AdministrativeSex', 'Code_value_general', 'SEX', 'PID-8', 'WARNING', 'Invalid_code');

//Race
for (j=0; j < ELRMessage.getRepeatCount(PID + '/Race'); j++) {
  fnCheckIfRecordReturned (false, PID + '/Race['+j+']/Identifier', 'V_Race_code', 'P_RACE', 'PID-10', 'INFO', 'Invalid_code');
}
//Race (end)

//PatientAddress - ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/StateOrProvince', 'State_code', 'STATE_CCD', 'PID-11.4', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/Country', 'Code_value_general', 'PSL_CNTRY', 'PID-11.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/AddressType', 'Code_value_general', 'EL_TYPE_PST_PAT', 'PID-11.7', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAddress[0]/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'PID-11.9', 'INFO', 'Invalid_code');
//PatientAddress (end)

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, PID + '/HomePhoneNumber[0]/TelecommunicationUseCode', 'Code_value_general', 'EL_USE', 'PID-13.2', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/HomePhoneNumber[0]/TelecommunicationEquipmentType', 'Code_value_general', 'EI_TYPE', 'PID-13.3', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BusinessPhoneNumber[0]/TelecommunicationUseCode', 'Code_value_general', 'EL_USE', 'PID-14.2', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BusinessPhoneNumber[0]/TelecommunicationEquipmentType', 'Code_value_general', 'EI_TYPE', 'PID-14.3', 'INFO', 'Invalid_code');

//ODS Comment: Not treated as repeatable.
fnCheckIfRecordReturned (false, PID + '/PrimaryLanguage[0]/Identifier', 'Language_code', 'P_LANG', 'PID-15.1', 'INFO', 'Invalid_code');

fnCheckIfRecordReturned (false, PID + '/MaritalStatus/Identifier', 'Code_value_general', 'P_MARITAL', 'PID-16.1', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientAccountNumber/IdentifierTypeCode', 'Code_value_general', 'EI_TYPE', 'PID-18.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/DriversLicenseNumberPatient/IssuingStateProvinceCountry', 'Code_value_general', 'EI_AUTH', 'PID-20.2', 'INFO', 'Invalid_code');

//ODS Comment: Presently NBS stores only one value

```

Javascript

```

fnCheckIfRecordReturned (false, PID + '/MothersIdentifier[0]/IdentifierTypeCode', 'Code_value_general', 'EI_TYPE', 'PID-21.5', 'INFO', 'Invalid_code');

fnCheckIfRecordReturned (false, PID + '/EthnicGroup/Identifier', 'Code_value_general', 'P_ETHN', 'PID-22.1', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/BirthPlace', 'Code_value_general', 'PSL_CNTRY', 'PID-23', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/MultipleBirthIndicator', 'Code_value_general', 'YNU', 'PID-24', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, PID + '/PatientDeathIndicator', 'Code_value_general', 'YNU', 'PID-30', 'INFO', 'Invalid_code');
//----- PID segment (end) -----

//----- NK1 segment -----
for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/PATIENT/NK1'); j++) {
    var NK1 = 'PATIENT_RESULT['+i+']/PATIENT/NK1['+j+']';

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Suffix', 'Code_value_general', 'P_NM_SFX', 'NK1-2.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Prefix', 'Code_value_general', 'P_NM_PFX', 'NK1-2.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/Degree_DEPRECATED', 'Code_value_general', 'P_NM_DEG', 'NK1-2.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Name[0]/NameTypeCode', 'Code_value_general', 'P_NM_USE', 'NK1-2.7', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, NK1 + '/Relationship/Identifier', 'Code_value_general', 'RL_TYPE', 'NK1-3', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/StateOrProvince', 'State_code', 'STATE_CCD', 'NK1-4.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/Country', 'Code_value_general', 'PSL_CNTRY', 'NK1-4.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/AddressType', 'Code_value_general', 'EL_TYPE_PST_PAT', 'NK1-4.7', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/Address[0]/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'NK1-4.9', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, NK1 + '/PhoneNumber[0]/TelecommunicationUseCode', 'Code_value_general', 'EL_USE', 'NK1-5.2', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, NK1 + '/PhoneNumber[0]/TelecommunicationEquipmentType', 'Code_value_general', 'EI_TYPE', 'NK1-5.3', 'INFO', 'Invalid_code');
}
//----- NK1 segment (end) -----

//----- ORDER_OBSERVATION group -----
for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) {
    var ORC = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/ORC';
    var OBR = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBR';
    //----- ORC segment -----
    //ODS Comment: Presently NBS stores only one value

```

Javascript

```

fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityName[0]/OrganizationNameTypeCode',
'Code_value_general', 'O_NM_USE', 'ORC-21.2', 'INFO', 'Invalid_code');

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/StateOrProvince', 'State
_code', 'STATE_CCD', 'ORC-22.4', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/Country', 'Code_value_gen
eral', 'PSL_CNTRY', 'ORC-22.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, ORC + '/OrderingFacilityAddress[0]/CountyParishCode', 'V
_State_county_code_value', 'COUNTY_CCD', 'ORC-22.9', 'INFO', 'Invalid_code');

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/StateOrProvince', 'State
_code', 'STATE_CCD', 'ORC-24.4', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/Country', 'Code_value_gen
eral', 'PSL_CNTRY', 'ORC-24.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, ORC + '/OrderingProviderAddress[0]/CountyParishCode', 'V
_State_county_code_value', 'COUNTY_CCD', 'ORC-24.9', 'INFO', 'Invalid_code');
//----- ORC segment (end) -----

//----- OBR segment -----
//-----OBR-4-----
//Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.
if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'L') {
  fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'Labtest
_loinc', '', 'OBR-4.1', 'WARNING', 'Invalid_local_code');
}
if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') ==
'L') {
  fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier',
'Labtest_loinc', '', 'OBR-4.4', 'WARNING', 'Invalid_local_code');
}

//Check LOINC codes against LOINC_condition and LOINC_code tables.
if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfCodingSystem') == 'LN') {
  fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'LOINC_code',
'', 'OBR-4.1', 'WARNING', 'Invalid LOINC_code');
  fnCheckIfRecordReturned (true, OBR + '/UniversalServiceIdentifier/Identifier', 'LOINC_con
dition', '', 'OBR-4.1', 'WARNING', 'Condition_not_found_for_LOINC');
}
if (ELRMessage.getField(OBR + '/UniversalServiceIdentifier/NameOfAlternateCodingSystem') ==
'LN') {
  fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier',
'LOINC_code', '', 'OBR-4.4', 'WARNING', 'Invalid LOINC_code');
  fnCheckIfRecordReturned (false, OBR + '/UniversalServiceIdentifier/AlternateIdentifier',
'LOINC_condition', '', 'OBR-4.4', 'WARNING', 'Condition_not_found_for_LOINC');
}
//-----OBR-4 (end)-----

//ODS Comment: Presently NBS stores only one value
fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Suffix', 'Code_value_general',
'P_NM_SFX', 'OBR-10.5', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Prefix', 'Code_value_general',
'P_NM_PFX', 'OBR-10.6', 'INFO', 'Invalid_code');
fnCheckIfRecordReturned (false, OBR + '/CollectorIdentifier[0]/Degree_DEPRECATED', 'Code
_value_general', 'P_NM_DEG', 'OBR-10.7', 'INFO', 'Invalid_code');

```

Javascript

```

    fnCheckIfRecordReturned (false, OBR + '/SpecimenActionCode', 'Code_value_general', 'M_HAN
DLING', 'OBR-11', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/DangerCode/Identifier', 'Code_value_general', 'M
_RISK', 'OBR-12.1', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, OBR + '/SpecimenSource[0]/SpecimenSourceNameOrCode/Identifier
', 'Code_value_general', 'SPECMN_SRC', 'OBR-15.1.1', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Suffix', 'Code_value_general', 'P
_NM_SFX', 'OBR-16.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Prefix', 'Code_value_general', 'P
_NM_PFX', 'OBR-16.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/OrderingProvider[0]/Degree_DEPRECATED', 'Code_value
_general', 'P_NM_DEG', 'OBR-16.7', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (true, OBR + '/ResultStatus', 'Code_value_general', 'ACT_OBJ_ST',
'OBR-25', 'WARNING', 'Invalid_code');

    for (k=0; k < ELRMessage.getRepeatCount(OBR + '/ResultCopiesTo'); k++) {
        fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Suffix', 'Code_value_general',
'P_NM_SFX', 'OBR-28.5', 'INFO', 'Invalid_code');
        fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Prefix', 'Code_value_general',
'P_NM_PFX', 'OBR-28.6', 'INFO', 'Invalid_code');
        fnCheckIfRecordReturned (false, OBR + '/ResultCopiesTo['+k+']/Degree_DEPRECATED', 'Code
_value_general', 'P_NM_DEG', 'OBR-28.7', 'INFO', 'Invalid_code');
    }

    for (k=0; k < ELRMessage.getRepeatCount(OBR + '/ReasonForStudy'); k++) {
        fnCheckIfRecordReturned (false, OBR + '/ReasonForStudy['+k+']/Identifier', 'Code_value_gen
eral', 'OBS_REASON', 'OBR-31.1', 'INFO', 'Invalid_code');
    }

    fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Suffix', 'Code_value
_general', 'P_NM_SFX', 'OBR-32.1.5', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Prefix', 'Code_value
_general', 'P_NM_PFX', 'OBR-32.1.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/Degree_DEPRECATED',
'Code_value_general', 'P_NM_DEG', 'OBR-32.1.7', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBR + '/PrincipalResultInterpreter/Name/AssigningAuthority
NamespaceID', 'Code_value_general', 'EI_AUTH', 'OBR-32.1.9', 'INFO', 'Invalid_code');
    //----- OBR segment (end) -----

    //----- OBSERVATION group -----

    for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBS
ERVATION'); k++) {
        var OBX = 'PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION['+k+']/OBX';

        //----- OBX segment -----

        //-----OBX-3-----

```

Javascript

```

//Translate any local codes to a LOINC code and check the resulting LOINC code against other
tables.
if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'L') {
    fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'Labtest_loinc',
'', 'OBX-3.1', 'WARNING', 'requiredField');
}
if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'L')
{
    fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'Lab
test_loinc', '', 'OBX-3.4', 'WARNING', 'Invalid_local_code');
}

//Check LOINC codes against LOINC_condition and LOINC_code tables.
if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfCodingSystem') == 'LN') {
    fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'LOINC_code', '',
'OBX-3.1', 'WARNING', 'Invalid LOINC code');
    fnCheckIfRecordReturned (true, OBX + '/ObservationIdentifier/Identifier', 'LOINC_con
dition', '', 'OBX-3.1', 'WARNING', 'Condition_not_found_for_LOINC');
}
if (ELRMessage.getField(OBX + '/ObservationIdentifier/NameOfAlternateCodingSystem') == 'LN')
{
    fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'LOINC
_code', '', 'OBX-3.4', 'WARNING', 'Invalid LOINC_code');
    fnCheckIfRecordReturned (false, OBX + '/ObservationIdentifier/AlternateIdentifier', 'LOINC
_condition', '', 'OBX-3.4', 'WARNING', 'Condition_not_found_for_LOINC');
}
//-----OBX-3 (end)-----

if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {
    //-----OBX-5-----
    //Handle OBX-5 variations based on the data type in OBX-2
    var OBX5DataType = ELRMessage.getField(OBX + '/ValueType');

    var requiredOBX5orOBX8 = false;
    if ((ELRMessage.getField(OBX + '/ObservationValue') == null || ELRMessage.getField(OBX + '/
ObservationValue') == '')
        && (ELRMessage.getField(OBX + '/AbnormalFlags[0]') == null || ELRMessage.getField(OBX +
'/AbnormalFlags[0]') == '')) {
        requiredOBX5orOBX8 = true;
    }
    else { requiredOBX5orOBX8 = false; }

    if ((OBX5DataType == 'CE') || (OBX5DataType == 'CWE')) {
        var OBX5CodingSys = ELRMessage.getField(OBX + '/ObservationValue/NameOfCodingSystem');
        if ((OBX5CodingSys == 'SN') || (OBX5CodingSys == 'SNM')) {
            fnCheckIfRecordReturned (true, OBX + '/ObservationValue/Identifier', 'Snomed_code', '',
'OBX-5.1', 'WARNING', 'Invalid_SNOMED_code');
            fnCheckIfRecordReturned (true, OBX + '/ObservationValue/Identifier', 'Snomed_condition',
'', 'OBX-5.1', 'WARNING', 'Condition_not_found_for_SNOMED');
        }
        var OBX5AltCodingSys = next.getField(OBX + '/ObservationValue/NameOfAlternateCodingSystem
');
        if ((OBX5AltCodingSys == 'SN') || (OBX5AltCodingSys == 'SNM')) {
            fnCreateXMLSNOMEDEntry (false, OBX + '/ObservationValue/AlternateIdentifier', 'Snomed
_code', '', 'OBX-5.4', 'WARNING', 'Invalid_SNOMED_code');
            fnCreateXMLSNOMEDEntry (false, OBX + '/ObservationValue/AlternateIdentifier', 'Snomed_con
dition', '', 'OBX-5.4', 'WARNING', 'Condition_not_found_for_SNOMED');
        }
    }
}

```

Javascript

```

    }
    if (OBX5DataType == 'SN') {
        fnCheckIfRecordReturned (false, OBX + '/ObservationValue/Comparator', 'Code_value_general', 'OV_NUM_CMP', 'OBX-5.1', 'INFO', 'Invalid_code');
        fnCheckIfRecordReturned (false, OBX + '/ObservationValue/SeparatorSuffix', 'Code_value_general', 'OV_NUM_SEP', 'OBX-5.3', 'INFO', 'Invalid_code');
    }
    //-----OBX-5 (end)-----

    //OBX-6 is required if OBX-2 is 'SN' or 'NM' and OBX-11 is not 'X'.
    fnCheckIfRecordReturned ((OBX5DataType == 'SN' || OBX5DataType == 'NM'), OBX + '/Units/Identifier', 'Code_value_general', 'UNIT_ISO', 'OBX-6.1', 'WARNING', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (requiredOBX5orOBX8, OBX + '/AbnormalFlags[0]/Identifier', 'Code_value_general', 'OBS_INTRP', 'OBX-8', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/ObservationResultStatus', 'Code_value_general', 'ACT_OBJ_ST', 'OBX-11', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/ProducersReference/NameOfCodingSystem', 'Code_value_general', 'EI_AUTH', 'OBX-15.3', 'INFO', 'Invalid_code');

    //ODS Comment: Presently NBS stores only one value
    fnCheckIfRecordReturned (false, OBX + '/ObservationMethod[0]/Identifier', 'Code_value_general', 'OBS_METH', 'OBX-17.1', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationName/IDNumber_DEPRECATED', 'Code_value_general', 'EI_AUTH', 'OBX-23.3', 'INFO', 'Invalid_code');

    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/StateOrProvince', 'State_code', 'STATE_CCD', 'OBX-24.4', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/Country', 'Code_value_general', 'PSL_CNTRY', 'OBX-24.6', 'INFO', 'Invalid_code');
    fnCheckIfRecordReturned (false, OBX + '/PerformingOrganizationAddress/CountyParishCode', 'V_State_county_code_value', 'COUNTY_CCD', 'OBX-24.9', 'INFO', 'Invalid_code');
    //----- OBX segment (end) -----
    } // -----(END) if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') (END)

    } // -----(END) for (k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION['+j+']/OBSERVATION'); k++) (END)

    //----- OBSERVATION group (end) -----

    } // -----(END) for (j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT['+i+']/ORDER_OBSERVATION'); j++) (END)
    //--- ORDER_OBSERVATION group (end) ---

    } // -----(END) for (i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
    //---- PATIENT_RESULT group (end) ----

    //
    //=====
    //_____ Build Output XML file (end)
    )
    //
    //=====

```

Javascript	
<code>next.text = dbInsertsXML;</code>	
Message Collector: Input Collation	
Collection Order	[ECLIR - ELR 2.3.1 Message Validation/68580cb2-5571-4fb6-b51d-a4e7796446af,ECLIR - ELR 2.3.1 Message Validation/6fd292f8-d669-44e7-ba29-2cf12f762ade]
Match Property	ELR_MESSAGE_ID
Message Set Timeout	4
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
Output Condition: Empty Message Body does not contain 'errorRecord' ignoring case	
 ECLIR - ELR Sink.....Sink	14
Error Connector	Page
 db Add Field Descriptions.....Database Message Extraction	402



db Add Field Descriptions

Database Message Extraction

On ECLIR - ELR 2.3.1 Message Validation route

Filter

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	

Configuration Properties	
Socket Timeout	0
Configuration File:	
<pre> <statement name="errorRecord" ide:type="executeSql" iterate="errorRecord" ide:summary="Adds the field description to each errorRecord node from the prior validations." ide:allowCompound Statements="true"> <sql>SELECT `/message/errorRecord/VALIDATION_TYPE` AS VALIDATION_TYPE , `/message/errorRecord/ELR_MESSAGE_ID` AS ELR_MESSAGE_ID , `/message/errorRecord/ERROR_CATEGORY` AS ERROR_CATEGORY , `/message/errorRecord/PROCESSING_STEP` AS PROCESSING_STEP , `/message/errorRecord/FIELD` AS FIELD , COALESCE((SELECT TOP(1) COLUMN_DESCRIPTION FROM ELR_HL7_FIELD_DESCRIPTIONS WHERE HL7_FIELD_NOTATION = `/message/errorRecord/FIELD` OR HL7_FIELD_NOTATION = LEFT(`/message/errorRecord/FIELD`, LEN(`/message/errorRecord/ FIELD`) - charindex('.', reverse(`/message/errorRecord/FIELD`))) OR HL7_FIELD_NOTATION = LEFT(`/message/errorRecord/FIELD`, LEN(`/message/errorRecord/ FIELD`) - (charindex('.', reverse(`/message/errorRecord/FIELD`), charindex('.', reverse(`/mes sage/errorRecord/FIELD`))+1))) ORDER BY HL7_FIELD_NOTATION DESC), NULL) AS FIELD_DESCRIPTION , `/message/errorRecord/FIELD_PATH` AS FIELD_PATH , `/message/errorRecord/FIELD_VALUE` AS FIELD_VALUE , `/message/errorRecord/ERROR_MSG` AS ERROR_MSG , `/message/errorRecord/ERROR_TYPE` AS ERROR_TYPE , `/message/errorRecord/ERROR_TYPE_DETAIL` AS ERROR_TYPE_DETAIL</sql> </statement> </config> </pre>	

Destinations

Name	Page
 no-op ELR Out to Errors Route..... No-operation	356



db ELR Validation Route Errors

Filter

Database LookUp

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: This Database Lookup filter inserts an error record into the ELR_ERROR table in the ECLIR database in the event an unhandled error is caught by the *ECLIR - ELR 2.3.1 Message Validation Route Error Handler*.

Configuration Properties		
Maximum Concurrency	1	
Database	[REDACTED]	
Host	[REDACTED]	
Port	[REDACTED]	
Database Name	[REDACTED]	
Username	[REDACTED]	
Password	[REDACTED]	
Additional Connection Properties	Property Name	Property Value
	[REDACTED]	[REDACTED]
Configuration File	[REDACTED]	
Message Type	Plain text	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:	[REDACTED]	

```

<statement name="ELR_MESSAGE ID Does Not exist" ide:type="executeSql" test="Null($ELR_MESSAGE_ID)" ide:summary="The ELR_MESSAGE ID does not exist in the ELR_MESSAGE table. Create an entry in that table to create an ID." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_MESSAGE (
    ORIGINAL_MESSAGE
  , [REDACTED]_ENVIRONMENT
  , SUCCESS_FL
  , INSERT_DATE
  )
VALUES (
  $messageContent
  , [REDACTED]_ENVIRONMENT
  , 0
  , SYSDATETIME()
  )</sql>
  <statement ide:type="compoundSqlStatement">
    <sql>SELECT \@ \@IDENTITY AS ELR_MESSAGE_ID</sql>
    <substituteColumn name="ELR_MESSAGE_ID">$ELR_MESSAGE_ID</substituteColumn>
  </statement>
  <statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull(@ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
    <sql>INSERT INTO ELR_ERROR (
      ELR_MESSAGE_ID
    , VALIDATION_TYPE
    , ERROR_CATEGORY
    , ERROR_TYPE
    , PROCESSING_STEP
    , ERROR_MSG

```

Configuration File: [REDACTED]

```
,INSERT_DATE
)
VALUES (
  @ELR_MESSAGE_ID
  , $VALIDATION_TYPE
  , $ERROR_CATEGORY
  , $ERROR_TYPE
  , $PROCESSING_STEP
  , $ERROR_MSG
  , SYSDATETIME()
) </sql>
</statement>
</statement>
<statement name="Insert into ELR_ERROR table" ide:type="executeSql" test="NotNull($ELR_MESSAGE_ID)" ide:summary="Add error record to the ELR_ERROR table." ide:allowCompoundStatements="true">
  <sql>INSERT INTO ELR_ERROR (
    ELR_MESSAGE_ID
    , VALIDATION_TYPE
    , ERROR_CATEGORY
    , ERROR_TYPE
    , PROCESSING_STEP
    , ERROR_MSG
    , INSERT_DATE
  )
  VALUES (
    $ELR_MESSAGE_ID
    , $VALIDATION_TYPE
    , $ERROR_CATEGORY
    , $ERROR_TYPE
    , $PROCESSING_STEP
    , $ERROR_MSG
    , SYSDATETIME()
  ) </sql>
</statement>
</config>
```

Destinations

Name	Page
 no-op REH to Sink..... No-operation	383



NBS Definition Validation

HL7 Acknowledgement Generation

On ECLIR - ELR 2.3.1 Message Validation route

Filter

Notes

Purpose: This HL7 Acknowledgement Generation filter creates an ACK message type containing errors listing any violations of the ECLIR_ELORU_2.3.1_NBS_validation.s3d message definition. These errors are parsed out in the JavaScript filter *js Set Message Error properties* and written to an XML flat file for writing the errors to the ECLIR ELR_ERROR table.

Notes

Configuration Properties

Message Definition	ECLIR_ELORU_2.3.1_NBS_validation.s3d	
Response Message Mapping		
Acknowledgement Mode	Validate, use Properties, and ACK	
NACK Mode	AR	
User Description		
On Message Validation Failure	Return Ack only to route	
On Successful Validation	Return Ack only to route	
Message Errors Property Name	MsgErrors	
HL7 Version	Same as input message	
Maintain Field Length	Ignore field lengths	
Fields to Copy	Source Field	Destination Field
	MSH.SendingApplication	MSH.ReceivingApplication
	MSH.SendingFacility	MSH.ReceivingFacility
	MSH.ReceivingApplication	MSH.SendingApplication
	MSH.ReceivingFacility	MSH.SendingFacility
	MSH.MessageControlId	MSA.MessageControlId
	MSH.ProcessingID	MSH.ProcessingID
Use Custom DateTime Format	false	

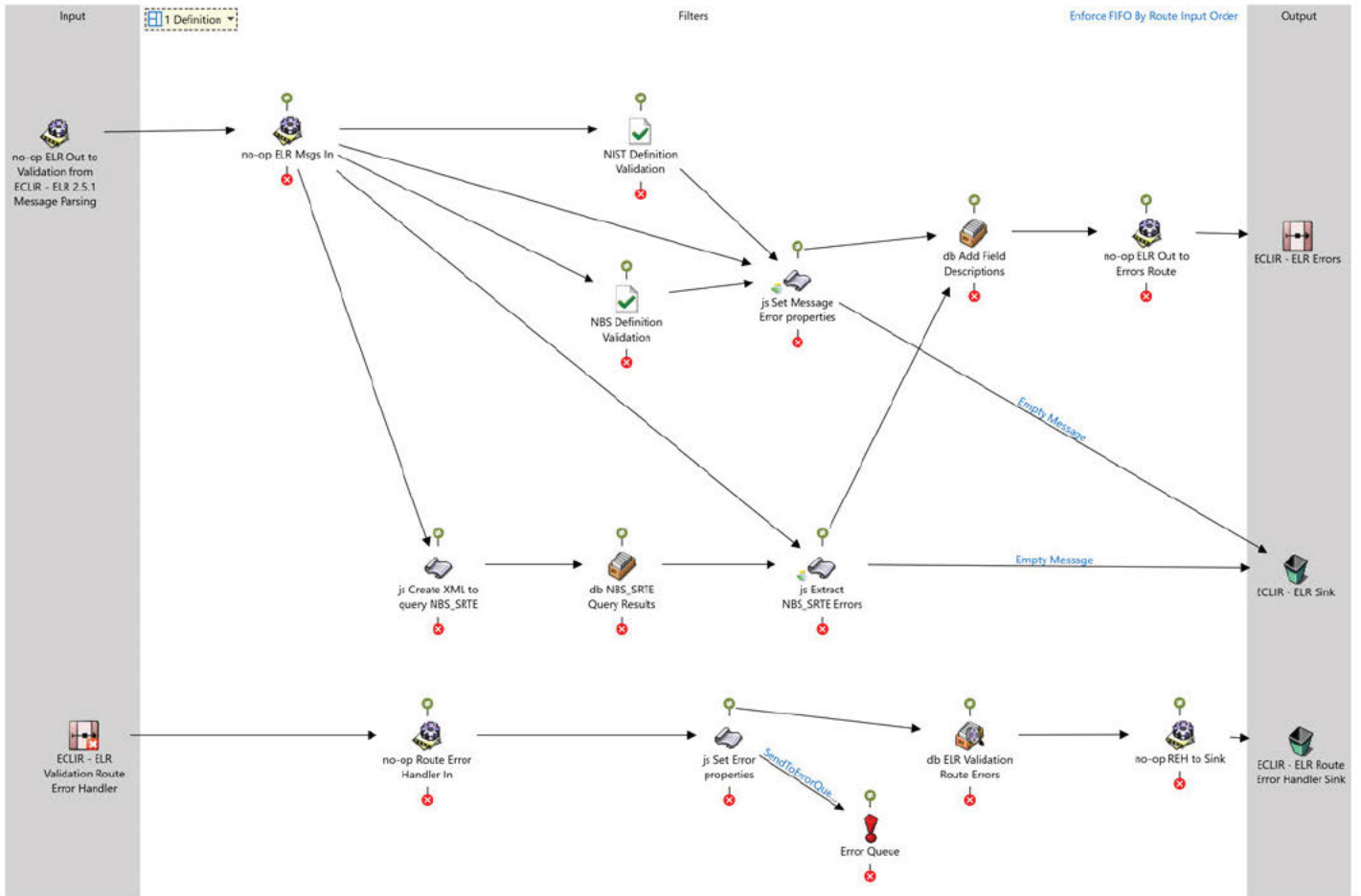
Destinations

Name	Page
js Set Message Error properties.....JavaScript (v.1)	357



ECLIR - ELR 2.5.1 Message Validation

Route



Notes

Purpose: This route receives HL7 v2.5.1 messages from the *ECLIR - ELR 2.5.1 Message Parsing* route. The data in the messages is validated against the following specifications and any violations will be sent to the *ECLIR - ELR Errors* route to be recorded in the ECLIR ELR_ERROR table:

ECLIR_ELROU_2.5.1_NIST_validation.s3d message definition

ECLIR_ELROU_2.5.1_NBS_validation.s3d message definition

NIST conditional predicate and conformance rules

NBS ODS processing information, conditional predicate and conformance rules

Inputs

no-op ELR Out to Validation (filter from "ECLIR - ELR 2.5.1 Message Parsing").....No-operation

Page

284

Outputs

ECLIR - ELR Errors.....Route

Page

157

Outputs	Page
 ECLIR - ELR Sink..... Sink	14
 ECLIR - ELR Route Error Handler Sink..... Sink	13

Filters

Name	Page
 db Add Field Descriptions..... Database Message Extraction	455
 db ELR Validation Route Errors..... Database LookUp	479
 db NBS_SRTE Query Results..... Database Message Extraction	451
 Error Queue..... Error Queue	451
 js Create XML to query NBS_SRTE..... JavaScript (v.1)	456
 js Extract NBS_SRTE Errors..... JavaScript (v.1)	467
 js Set Error properties..... JavaScript (v.1)	409
 js Set Message Error properties..... JavaScript (v.1)	411
 NBS Definition Validation..... HL7 Acknowledgement Generation	481
 NIST Definition Validation..... HL7 Acknowledgement Generation	482
 no-op ELR Msgs In..... No-operation	454
 no-op ELR Out to Errors Route..... No-operation	410
 no-op REH to Sink..... No-operation	456
 no-op Route Error Handler In..... No-operation	451

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



js Set Error properties

JavaScript (v.1)

On ECLIR - ELR 2.5.1 Message Validation route

Filter

Notes

Purpose: This JavaScript filter

Sets the property `VALIDATION_TYPE` with the type of validation error to be stored in the `VALIDATION_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `ERROR_CATEGORY` with the category of the error to be stored in the `ERROR_CATEGORY` column in the `ELR_ERROR` table. The category is set to 'FATAL' because the message stops processing.

Sets the property `ERROR_TYPE` with the type of error to be stored in the `ERROR_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `PROCESSING_STEP` with the Rhapsody route name and filter name to be stored in the `PROCESSING_STEP` column in the `ELR_ERROR` table.

Sets the property `ERROR_MSG` with the Rhapsody generated errors to be stored in the `ERROR_MSG` column in the `ELR_ERROR` table.

Sets the property `ENVIRONMENT` with the Rhapsody Variable `ECLIR_ENVIRONMENT` to be stored in the `ENVIRONMENT` column in the `ELR_MESSAGE` table.

Sets the content of the message back to the original to be stored in the `ORIGINAL_MESSAGE` column in the `ELR_MESSAGE` table if needed.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====

next.setProperty('VALIDATION_TYPE', 'Rhapsody');
next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
```

Javascript

```

next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - ELR 2.5.1 Message Validation (Route) ->'
+ next.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
continue: ' + next.getErrors());
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
.getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);

//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR
_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}

//This is a Rhapsody variable that should be set up during configuration.
var ENVIRONMENT = getVariable("ECLIR_ENVIRONMENT");
if (ENVIRONMENT == null || ENVIRONMENT == "") {
    ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('ENVIRONMENT', ENVIRONMENT);

// Load the snapshot back into the message.
if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
}

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (end
// _____
//
=====

```

Destinations

Name	Page
Output Condition	
SendToErrorQueue = 'true' ignoring case	
! Error Queue..... Error Queue	451
Error Connector	Page
db ELR Validation Route Errors..... Database LookUp	479

**no-op ELR Out to Errors Route**

Filter

No-operation

On ECLIR - ELR 2.5.1 Message Validation route

Notes

Purpose: This No-operation filter sends XML formatted error messages to the *ECLIR - ELR Errors* route.

Notes

Destinations

Name	Page
 no-op ELR Validation Errors In..... No-operation	159



js Set Message Error properties

Filter

JavaScript (v.1)

On ECLIR - ELR 2.5.1 Message Validation route

Notes

Purpose: In this JavaScript filter, each section of the JavaScript code traverses each segment and ordered group according to the message definition ECLIR_ELORU_2.5.1_NIST_validation.s3d for an ORU – Unsolicited Observation Message (Event R01). This message definition is used because it provides a less stringent structure than the ECLIR_ELORU_2.5.1_NBS_validation.s3d definition to allow for validation of repeating fields if present.

A message collector is used to input 3 messages into the filter.

The first message is the original HL7 v2 ELR message.

The second message is the HL7 v2 ACK message received from the *NIST Definition Validation* HL7 Acknowledgement Generation filter.

The third message is the HL7 v2 ACK message received from the *NBS Definition Validation* HL7 Acknowledgement Generation filter.

These 3 messages are then evaluated according to the following sections of code:

NIST definition errors. Any errors returned from the *NIST Definition Validation* HL7 Acknowledgement Generation filter are written to the output XML file to be stored in the ECLIR database.

NIST conformance and conditional predicate checks. Specific fields in the message are checked for conformance or presence if they are required because of a condition. Any violations are written to the output XML file to be stored in the ECLIR database.

NBS definition errors. Any errors returned from the *NBS Definition Validation* HL7 Acknowledgement Generation filter are written to the output XML file to be stored in the ECLIR database.

NBS processing information, conformance and conditional predicate checks. If a field is repeated but NBS only stores the first value, an informational error record is stored. Specific fields in the message are checked for conformance or presence if they are required because of a condition. Any violations are written to the output XML file to be stored in the ECLIR database.

Conformance and conditional predicate checks. Specific fields in the message are checked for conformance or presence if they are required because of a condition. Any violations are written to the output XML file to be stored in the ECLIR database.

This filter outputs an XML file that follows the ECLIR - Error.xsd format for use to write these errors to the ECLIR database in the *ECLIR - ELR Errors* route.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====
var next = output.append(input[0]);
var ELRMessage = input[0];
var ackNIST = input[1];
```

Javascript

```

var ackNBS = input[2];
var errList = '';
var dbInsertsXML = new XML(<message></message>);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ FUNCTIONS (start
)
//
=====
if (true) {
//FUNCTION: fnCreateXMLErrorRecord
//Function to create an XML node for writing an error record to the ELR_ERRORS table.
// recErrorTypeDetail parameter is used to combine errors into one record if the same rule was
// violated by multiple validations.
// Missing data
// Invalid date
// Invalid date/timeframe
// Invalid SSN
// Invalid format - LOINC
// Invalid format - date
// Invalid format - alpha
// Invalid format - numeric
// Invalid format - email
// Invalid format - zip
// Invalid format - ISO
// Invalid format - CLIA
// Invalid type - ISO
// Invalid type - CLIA
// Field must equal value
// Invalid value
// Field length
// Extra data
// Not unique
// Other
function fnCreateXMLErrorRecord (recValidationType, recErrorCategory, recFieldNotation, rec
PathHL7, recErrorMsg, recErrorType, recErrorTypeDetail) {
    tempEle = new XML(<errorRecord></errorRecord>);
    dbColumnName = <VALIDATION_TYPE>{fnTruncateToMax(recValidationType, 50)}</VALIDATION_TYPE>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <ELR_MESSAGE_ID>{next.getProperty('ELR_MESSAGE_ID')}</ELR_MESSAGE_ID>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <ERROR_CATEGORY>{fnTruncateToMax(recErrorCategory, 50)}</ERROR_CATEGORY>;
    tempEle.appendChild(dbColumnName);
    dbColumnName = <PROCESSING_STEP>{'Validation: ECLIR - ELR Validation (Route):js Set Message
Error properties (Filter)'}</PROCESSING_STEP>;
    tempEle.appendChild(dbColumnName);

```

Javascript

```

if (recFieldNotation != '') {
  dbColumnNode = <FIELD>{fnTruncateToMax(recFieldNotation, 150)}</FIELD>;
  tempEle.appendChild(dbColumnNode);
}
dbColumnNode = <FIELD_PATH>{fnTruncateToMax(recPathHL7, 1000)}</FIELD_PATH>;
tempEle.appendChild(dbColumnNode);
if (recPathHL7 != null && recPathHL7 != '') {
  var colFIELD_VALUE = ELRMessage.getField(recPathHL7);
  if (colFIELD_VALUE != null) {
    if (colFIELD_VALUE.length > 1000) {
      colFIELD_VALUE = '(**Field Value truncated. Please see original message.) ' + colFIELD
_VALUE;
    }
    dbColumnNode = <FIELD_VALUE>{fnTruncateToMax(colFIELD_VALUE, 1000)}</FIELD_VALUE>;
    tempEle.appendChild(dbColumnNode);
  }
}
dbColumnNode = <ERROR_MSG>{recErrorMsg}</ERROR_MSG>;
tempEle.appendChild(dbColumnNode);
dbColumnNode = <ERROR_TYPE>{fnTruncateToMax(recErrorType, 50)}</ERROR_TYPE>;
tempEle.appendChild(dbColumnNode);
dbColumnNode = <ERROR_TYPE_DETAIL>{fnTruncateToMax(recErrorTypeDetail, 75)}</ERROR_TYPE_DETAIL
>;
tempEle.appendChild(dbColumnNode);
dbInsertsXML.appendChild(tempEle);
}

//FUNCTION: fnTruncateToMax
//Function to truncate the colValue to the max length intMax.
function fnTruncateToMax(colValue, intMax) {
  if (colValue != null) {
    var strField = colValue.substring(0, intMax);
    return strField;
  }
}

//FUNCTION: fnCheckForSimilarErrors
//Function to check if a similar error was previously flagged for the same field.
function fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation, pathHL7, error
Msg, errorType, errorTypeDetail) {
  var otherErrors = false;
  //Check to see if a duplicate error was flagged by NIST or NBS validation.
  if (dbInsertsXML != '' && dbInsertsXML != null) {
    next.text = dbInsertsXML;
    if ((next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL
='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE") != null)) {
      var valType = next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR
_TYPE_DETAIL='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE");
      if (valType.indexOf(valDefinition) == -1) { valType += ', ' + valDefinition; }

      //Update validation type with new value concatenated.
      next.setField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL='" +
errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE", valType);

      //Check if the ERROR_CATEGORY is a higher severity. If so, set to higher severity.

```

Javascript

```

var valCategory = next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/ERROR_CATEGORY");
if (errorCategory != valCategory) {
    if ((errorCategory == 'FATAL') || (errorCategory == 'WARNING' && valCategory == 'INFO')) {

        next.setField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/ERROR_CATEGORY", errorCategory);
    }
}
otherErrors = true;
}
dbInsertsXML = next.xml;
}
if (!otherErrors) { fnCreateXMLErrorRecord(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, errorType, errorTypeDetail); }
}

//FUNCTION: fnValidateDataTypeRule
//Function to validate a field based on it's data type and conformance rules.
function fnValidateDataTypeRule (dataType, checkLOINC, dtValidationType, dtErrorCategory, dtFieldNotation, dtPathHL7) {
    switch(dataType) {
        case 'CWE_ELR':
            if (checkLOINC) {
                //Conformance Rule ELR-069:IF CWE.3 (Name of Coding System) value is "LN", SHALL be a valid LOINC code identifier format.
                if (ELRMessage.getField(dtPathHL7 + '/NameofCodingSystem') == 'LN') { fnValidateFormat(dtValidationType, dtErrorCategory, dtFieldNotation + '.1', dtPathHL7 + '/Identifier', '', '', '', 'LOINC'); }
                //Conformance Rule ELR-070:IF CWE.6 (Name of AlternateCoding System) value is "LN", SHALL be a valid LOINC code identifier format.
                if (ELRMessage.getField(dtPathHL7 + '/NameofAlternateCodingSystem') == 'LN') { fnValidateFormat(dtValidationType, dtErrorCategory, dtFieldNotation + '.4', dtPathHL7 + '/AlternateIdentifier', '', '', '', 'LOINC'); }
            }
            fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.3', dtFieldNotation + '.1', dtPathHL7 + '/NameofCodingSystem', dtPathHL7 + '/Identifier', '', 'IsAnyValue');
            fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.6', dtFieldNotation + '.4', dtPathHL7 + '/NameofAlternateCodingSystem', dtPathHL7 + '/AlternateIdentifier', '', 'IsAnyValue');
            fnRequiredIfTwoFieldsMissing (dtValidationType, dtErrorCategory, dtPathHL7 + '/OriginalText', dtPathHL7 + '/Identifier', dtPathHL7 + '/AlternateIdentifier', dtFieldNotation + '.9' + ' (OriginalText)', dtFieldNotation + '.1' + ' (Identifier)', dtFieldNotation + '.4' + ' (AlternateIdentifier)', dtFieldNotation + '.9');
            break;
        case 'EI_ELR':
            if (ELRMessage.getField(dtPathHL7 + '/UniversalIDType') == 'ISO') { fnValidateFormat(dtValidationType, dtErrorCategory, dtFieldNotation + '.3', dtPathHL7 + '/UniversalID', '', '', '', 'ISO'); }
            fnIsValidUniversalIDType(dtValidationType, dtErrorCategory, dtFieldNotation + '.4', dtPathHL7 + '/UniversalIDType', 'ISO');
            break;
        case 'HD_ELR':
            //Conformance Rule ELR-063:IF HD.3(Universal ID type) value is "ISO", SHALL be a valid ISO OID format.

```

Javascript

```

    if (ELRMessage.getField(dtPathHL7 + '/UniversalIDType') == 'ISO') { fnValidateFormat(dt
ValidationType, dtErrorCategory, dtFieldNotation + '.2', dtPathHL7 + '/UniversalID', '', '',
'', 'ISO'); }
    //Conformance Rule ELR-007:IF element is MSH-4.3 (Universal ID type) , then HD.3 (Universal
ID type) SHALL contain the value "ISO" OR "CLIA", ELSE HD.3 (Universal ID type) SHALL contain
the value "ISO"
    if (dtFieldNotation == 'MSH-4') {
        if (ELRMessage.getField(dtPathHL7 + '/UniversalIDType') == 'CLIA') { fnValidateFormat(dt
ValidationType, dtErrorCategory, dtFieldNotation + '.2', dtPathHL7 + '/UniversalID', '', '',
'', 'CLIA'); }
        fnIsValidUniversalIDType(dtValidationType, dtErrorCategory, dtFieldNotation + '.3', dt
PathHL7 + '/UniversalIDType', 'ISOCLIA');
    }
    else {
        fnIsValidUniversalIDType(dtValidationType, dtErrorCategory, dtFieldNotation + '.3', dt
PathHL7 + '/UniversalIDType', 'ISO');
    }
    break;
    case 'XCN_ELR':
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.9', dtField
Notation + '.1', dtPathHL7 + '/AssigningAuthority', dtPathHL7 + '/IDNumber', '', 'IsAnyValue');
        if (ELRMessage.getField(dtPathHL7 + '/AssigningAuthority') != null) { fnValidateData
TypeRule ('HD_ELR', false, dtValidationType, dtErrorCategory, dtFieldNotation + '.9', dtPathHL7
+ '/AssigningAuthority'); }
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.13', dtField
Notation + '.1', dtPathHL7 + '/IdentifierTypeCode', dtPathHL7 + '/IDNumber', '', 'IsAnyValue');
        if (ELRMessage.getField(dtPathHL7 + '/AssigningFacility') != null) { fnValidateData
TypeRule ('HD_ELR', false, dtValidationType, dtErrorCategory, dtFieldNotation + '.14', dt
PathHL7 + '/AssigningFacility'); }
        break;
    case 'XON_ELR':
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.1', dtField
Notation + '.10', dtPathHL7 + '/OrganizationIdentifier', dtPathHL7 + '/OrganizationName', '',
'IsNotValued');
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.6', dtField
Notation + '.10', dtPathHL7 + '/AssigningAuthority', dtPathHL7 + '/OrganizationIdentifier', '',
'IsAnyValue');
        if (ELRMessage.getField(dtPathHL7 + '/OrganizationIdentifier') != null) { fnValidate
DataRule ('HD_ELR', false, dtValidationType, dtErrorCategory, dtFieldNotation + '.6', dt
PathHL7 + '/AssigningAuthority'); }
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.7', dtField
Notation + '.10', dtPathHL7 + '/IdentifierTypeCode', dtPathHL7 + '/OrganizationIdentifier', '',
'IsAnyValue');
        break;
    case 'XTN_ELR':
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.4', dtField
Notation + '.7', dtPathHL7 + '/EmailAddress', dtPathHL7 + '/LocalNumber', '', 'IsNotValued');
        fnRequiredIfOtherField (dtValidationType, dtErrorCategory, dtFieldNotation + '.7', dtField
Notation + '.4', dtPathHL7 + '/LocalNumber', dtPathHL7 + '/EmailAddress', '', 'IsNotValued');
        break;
    case 'TDH_Required':
        var errorMsg = 'This message is missing ' + dtFieldNotation + ' (' + dtPathHL7.substr(dt
PathHL7.lastIndexOf('/')+1) + ') which is a required field.';

        if (ELRMessage.getField(dtPathHL7) == null) {
            fnCheckForSimilarErrors(dtValidationType, dtErrorCategory, dtFieldNotation, dtPathHL7,
errorMsg, 'CONFORMANCE', 'Missing data');
        }
    }

```

Javascript

```

    default:
        break;
    }
}

//FUNCTION: fnValidateFieldEqualsValue
//Function to check a conformance rule that a field equals a specific value.
function fnValidateFieldEqualsValue (valDefinition, errorCategory, fieldNotation, pathHL7, req
Value, fieldToEqualNotation, pathFieldToEqual) {
    var fieldEqualsValue = false;
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must be
';
    var reqValues;
    if (reqValue != null) {
        reqValues = reqValue.split('_||_');
        for (var i=0; i < reqValues.length; i++) {
            if (ELRMessage.getField(pathHL7) == reqValues[i]) { fieldEqualsValue = true; }
        }
    }
    else if (ELRMessage.getField(pathHL7) == null) { fieldEqualsValue = true; }

    if (!fieldEqualsValue) {
        if (fieldToEqualNotation != '') {
            errorMsg += 'the same value as ' + fieldToEqualNotation + ' (' + pathFieldToEqual.substr
(pathFieldToEqual.lastIndexOf('/')+1) + ')'. The ' + fieldNotation
+ ' value is "' + ELRMessage.getField(pathHL7) + '". The ' + fieldToEqualNotation + '
value is "' + ELRMessage.getField(pathFieldToEqual) + '".';
        }
        else {
            if (reqValue != null) {
                if (reqValues.length > 1) {
                    reqValue = reqValues.join(', ');
                    errorMsg += 'one of the values "' + reqValue + '". The value in this message is "' + ELRM
essage.getField(pathHL7) + '".';
                }
                else {
                    errorMsg += 'the value "' + reqValue + '". The value in this message is "' + ELRMessage
.getField(pathHL7) + '".';
                }
            }
        }
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Field must equal value');
    }
}

//FUNCTION: fnValidateDateTime
//Function to validate date/time according to the required format and to check the date is a
valid date.
function fnValidateDateTime (dtFormat, valDefinition, errorCategory, fieldNotation, pathHL7) {
    var fieldValue = Number(ELRMessage.getField(pathHL7)).toString();
    if (fieldValue != null) {
        var formatRep = '';
        var invalidDate = false;
        var invalidFormat = false;
    }
}

```

Javascript

```
switch(dFormat) {
    case 'TS_ELR_0000':
        if (ELRMessage.getField(pathHL7) != '0000') {
            if (fieldValue.match(/\\d{4}\\d{2}\\d{2}(\\d{2})?(\\d{2})?(\\d{2})?\\.?.?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
                formatRep = 'YYYYMMDD[HH[MM[SS[.S[S[S[S]]]]]]][+/-ZZZZ]';
                invalidFormat = true;
            }
            if (!invalidFormat && fieldValue.match(/(?:?:?:?:?:?:?:?:[13579][26]| [2468][048])00)|(?:?:[0-9]{2}?:?:[13579][26])|(?:?:[2468][048]|0[48]))(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01-9]|1[0-9]|2[0-9]))|(?:?:[0-9]{4})(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01][0-9]|2[0-8]))|(?:?:[0-9]|1[0-9]|2[0-3])(?:?:[0-5][0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
                invalidDate = true;
            }
        }
        break;
    case 'TS_ELR_SecOpt':
        if (fieldValue.match(/\\d{4}\\d{2}\\d{2}\\d{2}\\d{2}(\\d{2})?\\.?.?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
            formatRep = 'YYYYMMDDHHMM[SS[.S[S[S[S]]]]][+/-ZZZZ]';
            invalidFormat = true;
        }
        if (!invalidFormat && fieldValue.match(/(?:?:?:?:?:?:?:?:[13579][26]| [2468][048])00)|(?:?:[0-9]{2}?:?:[13579][26])|(?:?:[2468][048]|0[48]))(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01-9]|1[0-9]|2[0-9]))|(?:?:[0-9]{4})(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01][0-9]|2[0-8]))|(?:?:[0-9]|1[0-9]|2[0-3])(?:?:[0-5][0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
            invalidDate = true;
        }
        break;
    case 'TS_ELR_LD':
        if (fieldValue.match(/\\d{4}\\d{2}\\d{2}\\d{2}\\d{2}\\d{2}(\\+|-)[0-1][0-9]{3})?/) == null) {
            formatRep = 'YYYYMMDDHHMSS[.S[S[S[S[S]]]]][+/-ZZZZ]';
            invalidFormat = true;
        }
        if (!invalidFormat && fieldValue.match(/(?:?:?:?:?:?:?:?:[13579][26]| [2468][048])00)|(?:?:[0-9]{2}?:?:[13579][26])|(?:?:[2468][048]|0[48]))(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01-9]|1[0-9]|2[0-9]))|(?:?:[0-9]{4})(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01][0-9]|2[0-8]))|(?:?:[0-9]|1[0-9]|2[0-3])(?:?:[0-5][0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
            invalidDate = true;
        }
        break;
    case 'TS_ELR_YO':
        if (fieldValue.match(/\\d{4}(\\d{2})?(\\d{2})?(\\d{2})?(\\d{2})?(\\d{2})?\\.?.?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
            formatRep = 'YYYY[MM[DD[HH[MM[SS[.S[S[S[S]]]]]]]]][+/-ZZZZ]';
            invalidFormat = true;
        }
        if (!invalidFormat && fieldValue.match(/\\d{4}|(?:?:?:?:?:?:?:?:[13579][26]| [2468][048])00)|(?:?:[0-9]{2}?:?:[13579][26])|(?:?:[2468][048]|0[48]))(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01-9]|1[0-9]|2[0-9]))|(?:?:[0-9]{4})(?:?:?:?:?:?:?:[09|04|06|11])(?:?:[01-9]|1[0-9]|2[0-9]|30))|(?:?:[01|03|05|07|08|10|12])(?:?:[01-9]|1[0-9]|2[0-9]|3[01]))|(?:?:[02](?:?:[01][0-9]|2[0-8]))|(?:?:[0-9]|1[0-9]|2[0-3])(?:?:[0-5][0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
            invalidDate = true;
        }
        break;
```

Javascript

```

(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:(?:0[1-9]|2[0-8])))))(?:0[0-9]|1[0-9]|2[0-3])?(?:[0-5]
[0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
    invalidDate = true;
}
break;
case 'TS_ELR_TimeOpt':
    if (fieldValue.match(/\\d{4}\\d{2}\\d{2}(\\d{2})?(\\d{2})?(\\d{2})?\\.\\.\\d?\\d?\\d?(\\+|-)[0-1][0-9]
{3})?/) == null) {
        formatRep = 'YYYYMMDD[HH[MM[SS[.S[S[S[S]]]]]]][+/-ZZZZ]';
        invalidFormat = true;
    }
    if (!invalidFormat && fieldValue.match(/(?:?:?:?:?:?:?:[13579][26]|[2468][048])00|(?:?:[0-9]
){2}(?:?:[13579][26])|(?:[2468][048]|0[48])))(?:?:?:?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30
))|(?:?:01|03|05|07|08|10|12)(?:0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:0[1-9]|1[0-9]|2[0-9]))))|
(?:?:[0-9]{4}(?:?:?:09|04|06|11)(?:0[1-9]|1[0-9]|2[0-9]|30))|(?:?:01|03|05|07|08|10|12)(?:0[1
-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?:0[1-9]|2[0-8])))))(?:0[0-9]|1[0-9]|2[0-3])?(?:[0-5][0-9]
){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-)[0-1][0-9]{3})?/) == null) {
    invalidDate = true;
}
break;
}

if (invalidDate) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must
have a valid date. The value in this message "' + ELRMessage.getField(pathHL7) + '" is not a
valid date.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Invalid date');
}
if (invalidFormat) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must
follow the required date format of "' + formatRep + '". The value in this message "' + ELRM
essage.getField(pathHL7) + '" does not follow the correct format.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Invalid format - date');
}
}
}

//FUNCTION: fnValidateWithinTimeFrame
//Function to validate if date was within the valid time frame and not a future date.
function fnValidateWithinTimeFrame (valDefinition, errorCategory, fieldNotation, pathHL7, valid
TimeFrame) {
    var fieldDate = ELRMessage.getField(pathHL7);
    if ((fieldDate != null) && (fieldDate.length >= 8)) {
        fieldDate = new Date(Number(fieldDate.substr(0,4)),Number(fieldDate.substr(4,2))-1,Number
(fieldDate.substr(6,2)));
        // Calculate the date of the time frame.
        var todaysDate = new Date();
        todaysDate = new Date(todaysDate.getFullYear(),todaysDate.getMonth(),todaysDate.getDate());
        var pastDate = new Date(todaysDate);
        pastDate.setDate(pastDate.getDate() - validTimeFrame);
        if ((fieldDate < pastDate) || (fieldDate > todaysDate)) {

```

Javascript

```

    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') should
    be within the past ' + validTimeFrame +
    ' days and cannot be in the future. The value in this message "' + ELRMessage.getField
    (pathHL7) + '" is not within the correct time frame.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
    FORMANCE', 'Invalid date/timeframe');
  }
}

//FUNCTION: fnIsValidUniversalIDType
//Function to validate the correct Universal ID Type is present.
function fnIsValidUniversalIDType (valDefinition, errorCategory, fieldNotation, pathHL7,
  idType) {
  var errorMsg = 'The field ' + fieldNotation + ' (UniversalIDType) does not contain the correct
  value.';
  switch(idType) {
    case 'ISO':
      if (ELRMessage.getField(pathHL7) != 'ISO')
      {
        errorMsg += ' "ISO" is the only accepted value. The value in this message is "' + ELRM
        essage.getField(pathHL7) + '".';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
        'CONFORMANCE', 'Invalid type - ISO');
      }
      break;
    case 'ISOCLIA':

      errorMsg += ' "CLIA" or "ISO" are the only accepted values. The value in this message is
      "' + ELRMessage.getField(pathHL7) + '".';
      if ((ELRMessage.getField(pathHL7) != 'CLIA') && (ELRMessage.getField(pathHL7) != 'ISO')) {
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
        'CONFORMANCE', 'Invalid type - ISO/CLIA');
      }
      break;
  }
}

//FUNCTION: fnRequiredIfOtherField
//Function to check predicate condition for if a field is required based on another field's
value.
function fnRequiredIfOtherField (valDefinition, errorCategory, fieldNotation_ReqField, field
Notation_OtherField, path_ReqField, path_OtherField, otherSpecificValue, validationLogic) {
  var errorMsg = 'This message is missing ' + fieldNotation_ReqField + ' (' + path_ReqField
  .substr(path_ReqField.lastIndexOf('/')+1) + ')' + ' which is required if ' + fieldNotation
  _OtherField + ' (' + path_OtherField.substr(path_OtherField.lastIndexOf('/')+1) + ')';
  switch(validationLogic) {
    case 'IsAnyValue':
      //The field is required if the other field is any value.
      if (ELRMessage.getField(path_OtherField) != null && ELRMessage.getField(path_ReqField) ==
      null) {
        errorMsg += ' is valued.';
        fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
        Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
      }
      break;
  }
}

```